

Cover

Rule Configuration: Building a Least-Privilege Firewall Policy

9/1/2026

Instructor: Andry Rakotomalala

By: Dominic Yeakel

Part 1

Part 1: Orient Yourself and Establish the "As-Found" Baseline

Step 1: Confirm basic zones exist

Security Zones

+ Add

Name	Type	Interfaces	Zone Protection	User-ID
trust	Layer3	ethernet1/2, ethernet1/4	zone-protection-trust	✓
untrust	Layer3	ethernet1/1	zone-protection-untrust	
dmz	Layer3	ethernet1/3	zone-protection-dmz	

Step 2: Recreate the legacy rule

13	legacy-allow-all	trust	any	any	untrust	any	any	application-default	allow	—	0	    
----	------------------	-------	-----	-----	---------	-----	-----	---------------------	-------	---	---	---

Part 2

Part 2: Replace It With a Least-Privilege Rule Set

Step 1: Define what the branch segment actually needs

Service	Purpose	Ports
DNS	Name resolution for every other service to work	UDP/TCP 53
NTP	Accurate time — required for logs, certificates, and MFA to be trustworthy	UDP 123
HTTPS	Vendor portals, patch downloads, day-to-day browsing	TCP 443 (restrict to specific services using 443)
Core banking application	The branch app that tellers and bankers use all day	TCP 8443 (example)
Windows Update / patching	Keeps branch endpoints patched	TCP 443

Step 2: Build the specific allow rules

14	allow-dns	trust	any	any	untrust	any	any	dns	allow	—	0	    
15	allow-ntp	trust	any	any	untrust	any	any	ntp	allow	—	0	    
16	allow-web-https (ssl)	trust	any	any	untrust	any	ssl	application-default	allow	—	0	    
17	allow-web-https (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow	—	0	    
18	allow-banking-app	trust	any	any	untrust	Banking-App-Server	ssl	banking-app-8443	allow	—	0	    
19	allow-patching (ssl)	trust	any	any	untrust	any	ssl	application-default	allow	—	0	    
20	allow-patching (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow	—	0	    

Step 3: Remove the legacy rule and confirm default-deny

12	legacy-allow-all	trust	any	any	untrust	any	any	application-default	allow	—	0	    
13	allow-dns	trust	any	any	untrust	any	any	dns	allow	—	0	    
14	allow-ntp	trust	any	any	untrust	any	any	ntp	allow	—	0	    
15	allow-web-https (ssl)	trust	any	any	untrust	any	ssl	application-default	allow	—	0	    
16	allow-web-https (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow	—	0	    
17	allow-banking-app	trust	any	any	untrust	Banking-App-Server	ssl	banking-app-8443	allow	—	0	    
18	allow-patching (ssl)	trust	any	any	untrust	any	ssl	application-default	allow	—	0	    
19	allow-patching (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow	—	0	    
20	Default Deny	any	any	any	any	any	any	any	deny	—	34,521	    

Part 3

Part 3: Enable Application-Layer Awareness (App-ID)

13	allow-dns	trust	any	any	untrust	any	dns	application-default	allow	—	0	    
14	allow-ntp	trust	any	any	untrust	any	ntp	application-default	allow	—	0	    
15	allow-web-https (ssl)	trust	any	any	untrust	any	ssl	application-default	allow	—	0	    
16	allow-web-https (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow	—	0	    
17	allow-banking-app	trust	any	any	untrust	Banking-App-Server	ssl	banking-app-8443	allow	—	0	    
18	allow-patching (ssl)	trust	any	any	untrust	any	ssl	application-default	allow	—	0	    
19	allow-patching (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow	—	0	    

For the “allow-banking-app” rule, this rule is effectively port/protocol-based (TCP 8443, SSL) rather than application-specific, since no custom App-ID signature was available for the banking application. Risk: any other service that also uses TLS on port 8443 to the same destination IP would also match this rule and be allowed, even if it isn't the intended banking application.

Part 4

Part 4: Enable Threat Prevention and Threat Intelligence

Step 1: Build a Threat Prevention profile

Vulnerability Profiles		+ Add
Name	Description	
Vuln-Protect-Default	Default vulnerability protection	

Profile not editable in simulator, using default

Step 2: Attach it to your rules

Security Policy Rule — allow-dns

General

Source

Destination

Application

Service/URL Category

Actions

Target

Action

Allow

Profile Setting

Profiles

Security Profile Group

Vuln-Protect-Default

Log at Session Start

☒ Enable

Log at Session End

☒ Enable

Log Forwarding

default

Schedule

None

QoS Marking

None

Disable Server Response Inspection

☐ Disable

OK

Cancel

Added security profile group “Vuln-Protect-Default” to rule “allow-dns” and repeated for all allow policies

13	allow-dns	trust	any	any	untrust	any	dns	application-default	allow		0				
14	allow-ntp	trust	any	any	untrust	any	ntp	application-default	allow		0				
15	allow-web-https (ssl)	trust	any	any	untrust	any	ssl	application-default	allow		0				
16	allow-web-https (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow		0				
17	allow-banking-app	trust	any	any	untrust	Banking-App-Server	ssl	banking-app-8443	allow		0				
18	allow-patching (ssl)	trust	any	any	untrust	any	ssl	application-default	allow		0				
19	allow-patching (web-browsing)	trust	any	any	untrust	any	web-browsing	application-default	allow		0				

The green shields show the security profile group is configured for all allow policies

Step 3: Write two sentences: what an external dynamic list does, and why a feed that silently stops updating is a security gap nobody notices.

An External Dynamic List lets a firewall pull an outside feed of IPs, URLs, or domains and automatically use it in policy, so blocklists and allowlists stay current without manual rule edits. The risk with a silently stalled feed is that the firewall keeps enforcing the last version it downloaded and shows no error, so rules that look active are actually running on stale data while new threats slip through, or legitimate traffic gets blocked by outdated entries, and nothing in the UI signals that anything is wrong.

Part 5

Part 5: Test That Required Traffic Passes and Everything Else Is Blocked

#	Traffic	Expected Result	Actual Result
1	trust → untrust, DNS (UDP 53)	Allowed	Rule 13 (allow-dns). Matches app dns, service application-default. Allowed
2	trust → untrust, NTP (UDP 123)	Allowed	Rule 14 (allow-ntp). Matches app ntp, service application-default. Allowed
3	trust → untrust, HTTPS (TCP 443)	Allowed	Rule 15 (allow-web-https (ssl)). Matches app ssl, service application-default (443 is ssl's default port). Allowed
4	trust → untrust, banking app port	Allowed	Rule 17 (allow-banking-app). Allowed only if destination matches subnet defined under Banking-App-Server
5	trust → untrust, unrelated port (e.g. TCP 23 / Telnet)	Blocked	Falls to implicit deny-all. Blocked
6	trust → untrust, unrelated port (e.g. TCP 3389 / RDP)	Blocked	Falls to implicit deny-all. Blocked
7	untrust → trust, any traffic (unsolicited inbound)	Blocked	Falls to implicit deny-all. Blocked

Reflection Questions

Reflection Questions:

1. Why does a rule like legacy-allow-all tend to survive for years in a real network even though everyone agrees it's a bad idea?

A rule like legacy-allow-all tends to survive for years because nobody wants to be the person who breaks production by tightening it, since a broad allow-all rule almost certainly has unknown legitimate traffic quietly riding on it. Removing or narrowing it requires someone to first identify everything currently depending on it, which takes time and coordination that rarely gets prioritized over more urgent work. It also often predates the current team, so there's no one confident enough in its full blast radius to safely change it. The path of least resistance is always to leave it alone and build new, tighter rules alongside it rather than touch the existing one.

2. What specifically can App-ID catch that a traditional port-and-protocol firewall rule would miss?

A port-and-protocol rule only checks "is this TCP 443" which any application can use. App-ID actually inspects the traffic's behavior and signatures to confirm it's genuinely the application it claims to be. This catches things like a tunneling tool or unauthorized app disguising itself over port 443 to blend in with normal HTTPS traffic, which a port-based rule would wave through without question.

3. Your default-deny rule and your specific allow rules both stop unwanted traffic. Why do you still need both, instead of relying on just one or the other?

Default-deny acts as a safety net as it guarantees that anything not explicitly planned for is blocked, so a misconfiguration, an overlooked service, or a new attack technique doesn't get through by accident. But default-deny alone gives you no visibility since it can't tell you which allowed traffic is expected versus suspicious, and it can't enforce that your DNS traffic is only DNS or your banking app traffic is only going to the right server. Specific allow rules give you that precision and auditability, defining exactly what's permitted and why. The allow rules do the fine-grained enforcement while default-deny catches everything those rules didn't anticipate.